Sistemas Operacionais - I

Prof. Cláudio Luís V. Oliveira
claudio.oliveira@fatec.sp.gov.br



1

Sistemas Operacionais



- Um sistema operacional é formado por um conjunto de rotinas que oferece serviços aos usuários e às suas aplicações;
- Esse conjunto de rotinas recebe o nome de núcleo do sistema ou kernel;
- A estrutura do núcleo, ou seja, o modo como o código do sistema está organizado e o relacionamento entre os seus diversos componentes, pode variar de acordo com a concepção de cada sistema operacional.

2

Sistemas Operacionais



- Existem três maneiras dos usuários se comunicarem com o kernel:
- Através das chamadas de rotinas do sistema realizadas pelas aplicações;
- Por meio de utilitários;
- Ou através de linguagens de comandos.

3

Sistemas Operacionais



- Diferente de uma aplicação convencional, que apresenta seqüências de início, meio e fim, as rotinas de um sistema operacional são executadas concorrentemente, sem apresentar uma ordem predefinida;
- A execução ocorre com base em eventos dissociados do tempo (eventos assíncronos);
- Muitos desses eventos estão relacionados ao hardware e a tarefas internas do próprio sistema operacional.

4

Sistemas Operacionais



- Principais funções do núcleo:
- Tratamento de interrupções e exceções;
- Criação e remoção de processos e threads;
- Sincronização e comunicação entre processos e threads;
- Escalonamento e controle de processos e threads;
- Gerência de memória.

5

Sistemas Operacionais



- Principais funções do núcleo (continuação):
- Gerência do sistema de arquivos;
- Gerência dos dispositivos de Entrada e Saída;
- Suporte a redes locais e distribuídas;
- Registros, auditoria e segurança de uso do sistema.

6

Sistemas Operacionais



- Uma das principais características dos sistemas operacionais multiprogramáveis é permitir que vários programas compartilhem o processador;
- O sistema operacional deve ser responsável pelo controle da utilização da UCP, de forma a impedir que algum programa utilize-a de forma inadequada.

7

Sistemas Operacionais



- Como vários programas ocupam a memória simultaneamente, cada usuário deve possuir uma área reservada onde seus dados e programas são armazenados;
- De modo similar a memória, um disco pode armazenar arquivos de diferentes usuários;
- O sistema operacional deve garantir a integridade e a confidencialidade dos dados, permitindo que dois ou mais usuários tenham acesso simultâneo a um mesmo arquivo.

8

Sistemas Operacionais



- Uma preocupação nos projetos dos sistemas operacionais é a proteção ao núcleo do sistema e de acesso aos seus serviços;
- Normalmente utiliza-se um mecanismo presente no hardware dos processadores conhecido como *modo de acesso*;
- Em geral, os processadores apresentam dois modos de acesso: *modo usuário* e *modo kernel*.

9

Sistemas Operacionais

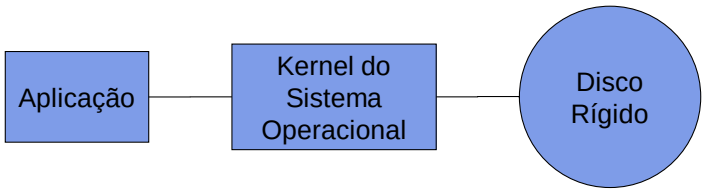


- Quando o processador está no modo usuário, uma aplicação apenas pode executar um grupo de instruções conhecidas como não-privilegiadas;
- Por outro lado, no modo kernel, a aplicação pode ter acesso ao conjunto completo de instruções do processador;
- Uma aplicação, por si só, não pode definir o modo de acesso, sendo esta responsabilidade do sistema operacional.

10

Sistemas Operacionais

- Por exemplo, quando uma aplicação precisa acessar o disco rígido ela precisará recorrer ao sistema operacional para realizar esta tarefa:



11

Sistemas Operacionais

- As *rotinas do sistema operacional* compõem o núcleo, oferecendo serviços aos usuários e aplicações;
- Todas as funções do núcleo são implementadas por rotinas que possuem em seu código instruções privilegiadas (modo kernel);
- Todo o controle de execução de rotinas do sistema operacional é realizado pelo mecanismo conhecido como *system call* (chamada ao sistema).

12

Sistemas Operacionais



- Toda vez que uma aplicação desejar chamar uma rotina do sistema operacional, o mecanismo *system call* é ativado;
- Inicialmente o sistema operacional verificará se a aplicação possui privilégios para executar a rotina solicitada;
- Em seguida, o sistema operacional salva o conteúdo dos registradores, altera o modo de acesso do processador de usuário para kernel e executa a rotina.

13

Sistemas Operacionais

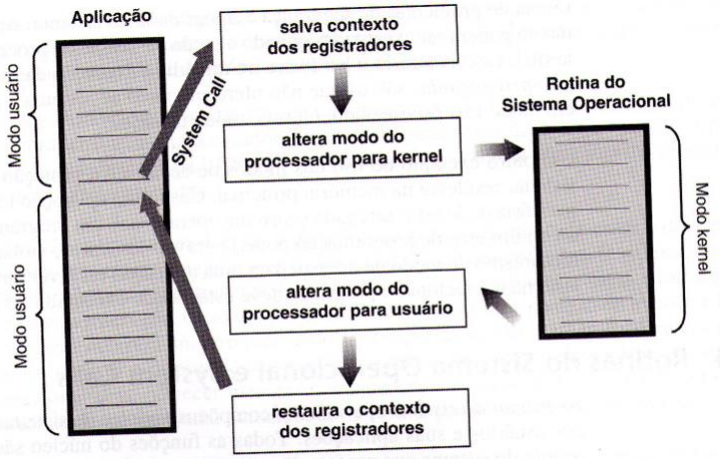


- Ao término da execução da rotina, o processador volta ao modo usuário e o conteúdos dos registradores é recuperado;
- A aplicação pode continuar a sua execução em modo usuário;
- Uma aplicação apenas pode ser executada com o processador em modo usuário, caso contrário, um *mecanismo de proteção por hardware* garantirá a segurança do sistema.

14

Sistemas Operacionais

- Chamada a uma rotina do sistema:



15

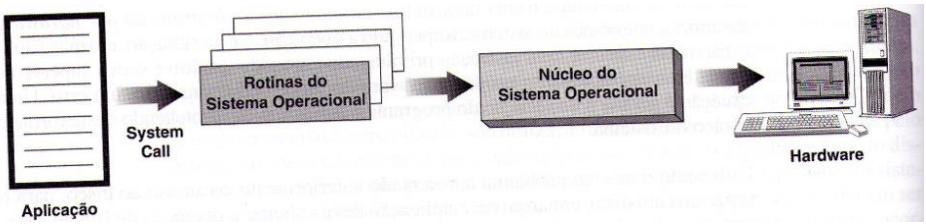
Sistemas Operacionais

- As *rotinas do sistema* e o *mecanismo de system call* podem ser entendidos como uma porta de entrada para o núcleo do sistema operacional e aos seus serviços;
- Quando uma aplicação desejar um serviço do sistema, deverá realizar a chamada da rotina através de uma system call;
- Conforme os parâmetros fornecidos, a solicitação é processada e a resposta é retornada a aplicação.

16

Sistemas Operacionais

- System Call:



- O termo system call é tipicamente utilizado em sistemas Unix, porém outros sistemas apresentam o mesmo conceito, apenas usando nomes diferentes:

17

Sistemas Operacionais

- O termo system call é tipicamente utilizado em sistemas Unix, porém outros sistemas apresentam o mesmo conceito, apenas usando nomes diferentes, por exemplo:
- System Services (OpenVMS);
- Application Program Interface – API (Windows).

18

Sistemas Operacionais



- Funções das system calls:

Funções	System calls
Gerência de processos e threads	Criação e eliminação de processos e threads Alteração das características de processos e threads Sincronização e comunicação entre processos e threads Obtenção de informações sobre processos e threads
Gerência de memória	Alocação e desalocação de memória
Gerência do sistema de arquivos	Criação e eliminação de arquivos e diretórios Alteração das características de arquivos e diretórios Abrir e fechar arquivos Leitura e gravação em arquivos Obtenção de informações sobre arquivos e diretórios
Gerência de dispositivos	Alocação e desalocação de dispositivos Operações de entrada/saída em dispositivos Obtenção de informações sobre dispositivos

19

Sistemas Operacionais



- Exemplo de utilização da Application Program Interface (API) do Windows:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{ time_t agora;
  time(&agora);
  printf("%s", ctime(&agora));
  system("PAUSE");
  return 0;
}
```

System Call

20

Sistemas Operacionais



- Cada sistema operacional possui o seu próprio conjunto de rotinas, com nomes, parâmetros e formas de ativação específicos;
- Desta forma, aplicações escritas em um sistema operacional não pode ser portada diretamente para um outro sistema, exigindo alterações no código fonte.

21

Sistemas Operacionais



- Uma tentativa de criação de uma biblioteca de chamadas objetivando uma padronização foi elaborada pelos institutos ISO e IEEE, resultando em um conjunto chamado como *POSIX (Portable Operating System Interface for Unix)*;
- O POSIX estabeleceu uma biblioteca padrão e as aplicações desenvolvidas no modelo de camadas do POSIX, podem ser portadas entre diferentes sistemas.

22

Sistemas Operacionais



- Apesar de inicialmente ser voltado para a unificação das diferentes versões do UNIX, atualmente a maioria dos sistemas operacionais modernos oferece algum tipo de suporte ao POSIX como, por exemplo:
  - Windows (Microsoft);
  - AIX (IBM);
  - HP-UX (HP);
  - Solaris (Sun).

23

Sistemas Operacionais



- A *linguagem de comando* ou *linguagem de controle*, permite que o usuário se comunique de uma forma simples com o sistema operacional;
- Possibilita ao usuário executar tarefas específicas do sistema como, por exemplo, criar, ler ou eliminar arquivos ou pastas e verificar a data e hora do sistema;
- A linguagem de comando permite uma interface direta com o sistema operacional.

24

Sistemas Operacionais



- Cada sistema operacional possui a sua própria linguagem de comando como:
  - Digital Command Language – DCL (OpenVMS);
  - Job Control Language – JCL (MVS-IBM);
  - Comandos do shell (Unix);
  - Comandos do Windows (herdados do MS-DOS).

25

Sistemas Operacionais



- Exemplos de comandos no Windows:

Comando	Descrição
dir	Exibe o conteúdo de uma pasta
cd	Altera a pasta de trabalho
type	Exibe o conteúdo de um arquivo
del	Apaga arquivos
mkdir	Cria uma pasta
ver	Mostra a versão do Windows

26

Sistemas Operacionais



- Exemplos de comandos do shell no Unix:

Comando	Descrição
ls	Exibe o conteúdo de uma pasta
cd	Altera a pasta de trabalho
more	Exibe o conteúdo de um arquivo
rm	Apaga arquivos
mkdir	Cria uma pasta
uname	Mostra a versão do Unix

27

Sistemas Operacionais



- Um comando, após digitado pelo usuário, é interpretado pelo *shell* ou *interpretador de comandos*, que irá verificar a sintaxe, realizar as chamadas às rotinas do sistema operacional e, ao final, mostrar uma mensagem informativa;
- Em geral, o shell não faz parte do kernel do sistema operacional, possibilitando a flexibilidade na criação de diferentes linguagens, por exemplo, o Unix oferece três interpretadores de comandos: Bourne Shell, C Shell e Korn Shell.

28

Sistemas Operacionais

- Na maioria dos sistemas operacionais, as linguagens de comando evoluíram no sentindo de permitir a interação mais amigável com o usuário, utilizando interfaces gráficas contendo janelas e ícones entre outros elementos.



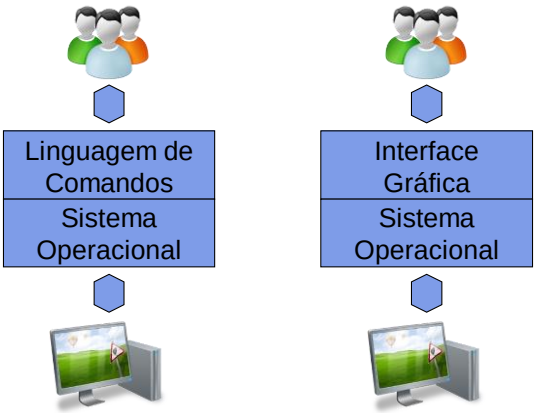
Evolução



29

Sistemas Operacionais

- Possibilidades de interface entre o usuário e o sistema operacional:



30

Sistemas Operacionais

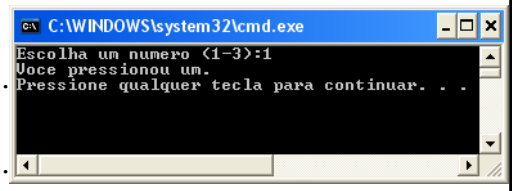
- As *linguagens de comandos* oferecem a possibilidade de criação de programas contendo estruturas de decisão e interação;
- Esses programas são uma seqüência de comandos armazenados em um arquivo texto, denominados *arquivos de comando*, *arquivos batch* ou *shell scripts*;
- Um dos principais motivos para seu uso é automatizar tarefas ligadas à gerência do sistema, como, por exemplo, os backups.

31

Sistemas Operacionais

- Exemplo de um arquivo batch do Windows:

```
@ECHO OFF
SET /P entrada=Escolha um numero (1-3):
IF %entrada% == 3 GOTO Tres
IF %entrada% == 2 GOTO Dois
IF %entrada% == 1 GOTO Um
GOTO Fim
:Tres
  ECHO Voce pressionou tres.
  GOTO Fim
:Dois
  ECHO Voce pressionou dois.
  GOTO Fim
:Um
  ECHO Voce pressionou um.
:Fim
```



32

Sistemas Operacionais



- Quanto um computador é ligado não há sistema operacional carregado na sua memória;
- Desta forma, os componentes do sistema operacional devem ser carregados para a memória principal toda vez que o computador é ligado;
- Este procedimento é denominado *ativação do sistema* ou *boot*.

33

Sistemas Operacionais



- O procedimento de ativação inicia-se com a execução de um programa chamado *boot loader*, que está localizado em um endereço físico da memória ROM;
- Este programa chama a execução de outro programa chamado POST (Power-On Self Test), que identifica possíveis falhas no hardware básico do computador;
- Em seguida, o procedimento de ativação procura por um sistema operacional em um dos dispositivos de armazenamento.

34

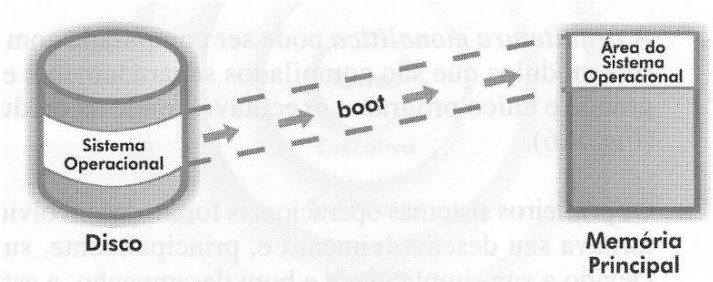
Sistemas Operacionais

- Um conjunto de instruções que está carregado em um bloco específico do dispositivo (boot sector) é carregado para a memória principal;
- A partir da execução deste conjunto de instruções o sistema operacional é finalmente carregado para a memória principal.

35

Sistemas Operacionais

- Além do processo de carga, a ativação do sistema também consiste em executar os arquivos de inicialização, que são responsáveis pela customização e configuração de hardware e software específicos.



36